# Qbase Ephemeral Relay Messaging Plan

This document outlines the architecture, rationale, and modifications implemented to establish a secure, WhatsApp-style ephemeral relay messaging system in Qbase. Under this framework, the Appwrite remote database functions strictly as a temporary store-and-forward relay. Once a message is securely delivered, decrypted, and saved inside the receiver's local Room database, it is instantly deleted from Appwrite, leaving zero data footprint in the cloud.

## 1. Limitations of the Old Architecture

Our technical research identified critical bugs that previously prevented ephemeral messaging from functioning:

• **Local Deletion Propagation Bug**: The real-time observers listened to Appwrite delete events and deleted the local Room records. When the receiver cleared their cloud footprint, the sender's copy was deleted too.
• **Incomplete Offline/Historical Cleanup**: Historical offline messages fetched via `listDocuments()` when coming online were never cleared from Appwrite.
• **Incomplete Background Cleanup**: Messages delivered globally in the background inside `observeAllIncomingMessages()` were written to Room but left on the server indefinitely.

---

## 2. Ephemeral Relay Design Principles

To match the security and architecture of industry standards (like WhatsApp):

1. **Relay Only**: The Appwrite server never acts as a permanent message archive.
2. **Asymmetric Key Encryption**: Payloads are E2EE encrypted; only the recipient can decrypt the session key.
3. **Instant Erasure**: The recipient assumes full responsibility for erasing the remote document the millisecond local delivery succeeds.
4. **Local Sovereignty**: Client local Room database records are permanent and are completely immune to server-side delete events.

---

## 3. Implemented Modifications

We have successfully integrated the following self-healing modifications in `SyncRepository.kt`:

### 3.1 Removing Deletion Propagation

We modified the real-time subscriptions in both `observeAndSyncMessages` and

`observeAllIncomingMessages` to ignore `.delete` broadcasts. Local Room DB records are now preserved when cloud copies are deleted.

### 3.2 Clearing Historical Offline Messages

We added a delete-on-delivery trigger during the initial sync load. As soon as offline messages are written to Room, they are cleared from the Appwrite cloud.

### 3.3 Clearing Background Delivered Messages

We added an immediate background deletion trigger after background-delivered messages are written to Room, ensuring zero leftover server footprint.